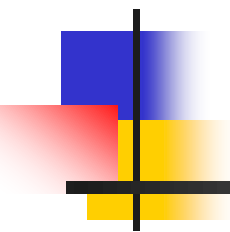
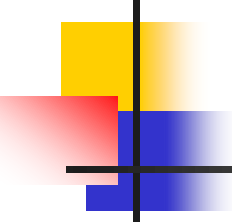


DATA STRUCTURES AND ALGORITHMS



Lecture Notes 8.1

Pradit Pitaksathienkul



ROAD MAP

- **PRIORITY QUEUES (HEAPS)**
 - Model
 - Simple Implementations
 - Binary Heap
 - **Insert**
 - **deleteMin**

```

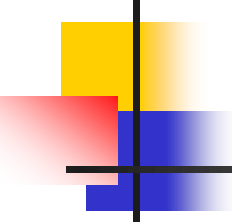
class BinaryHeap:
def __init__(self, capacity=100):
# สร้าง list ที่มีขนาด capacity + 1 (index 0 ไม่ถูกใช้งาน)
    self.array = [None] * (capacity + 1)
    self.currentSize = 0

def is_empty(self):
    return self.currentSize == 0

def is_full(self):
    return self.currentSize == len(self.array) - 1

def find_min(self):
    if self.is_empty():
        print("Queue is empty")
        return None
    return self.array[1]

```



Basic Heap Operations - **insert**

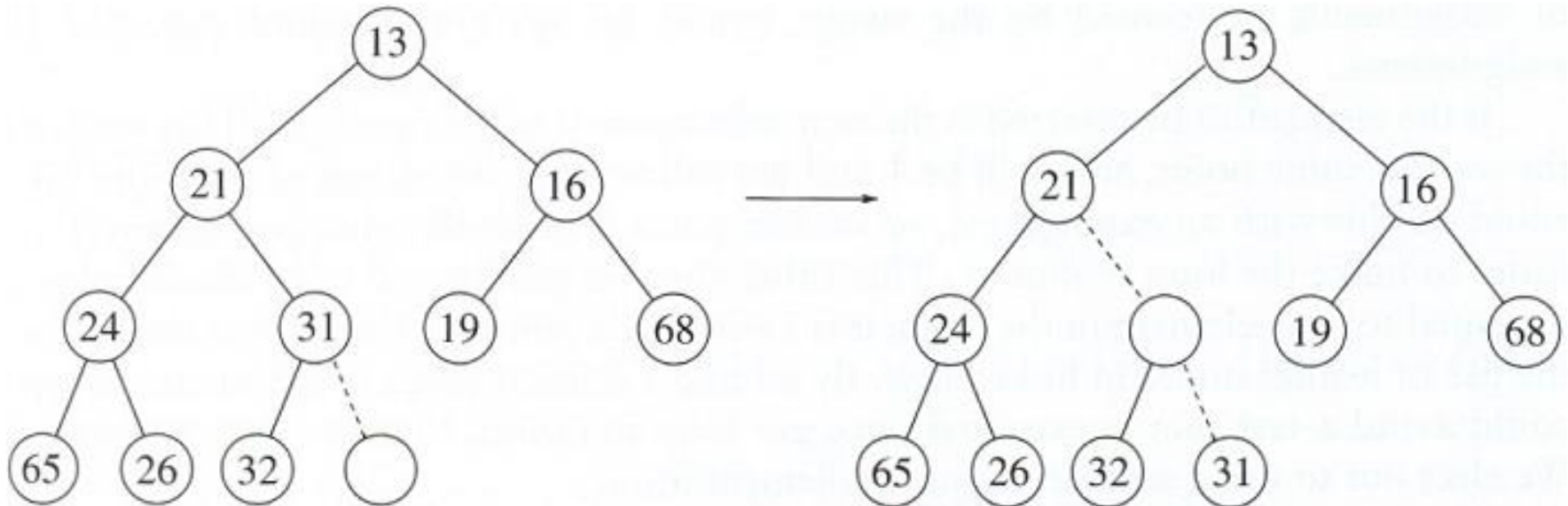
Algorithm

- create a hole in the next available array position
- while inserting X in the hole violate heap order
 - Percolate up the hole (by swapping with the parent)
- insert X in the hole

Example

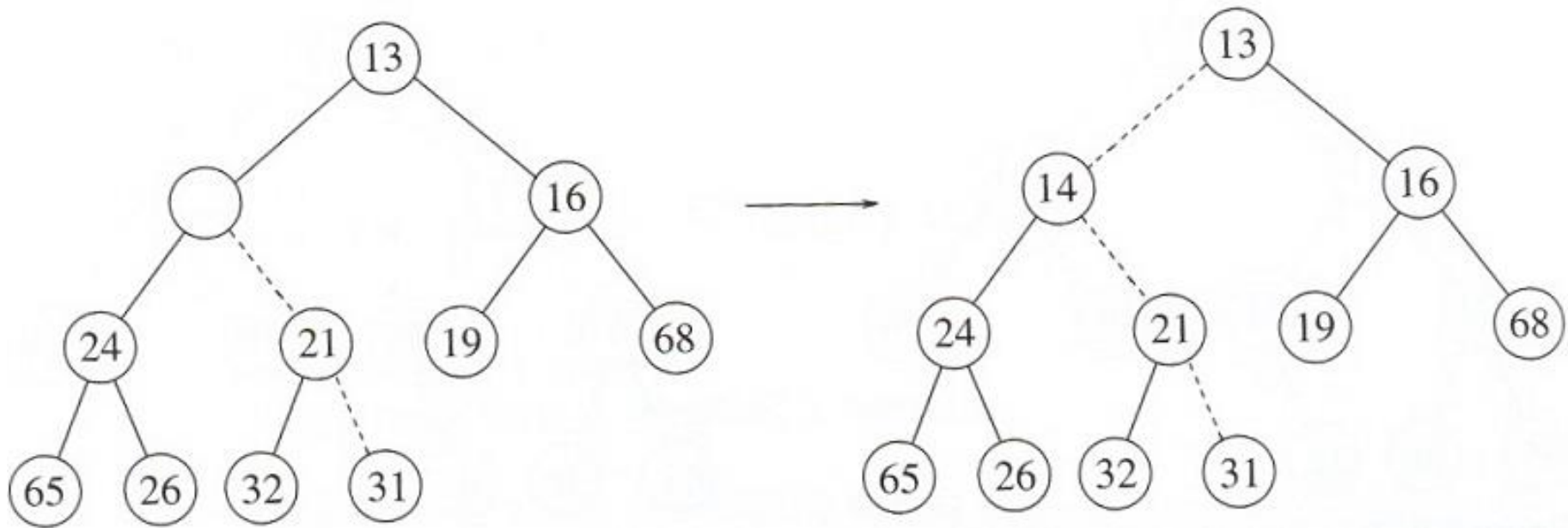
Attempt to insert 14

- Creating a **hole**
- Bubbling the hole up



Example

Remaining two steps to insert 14



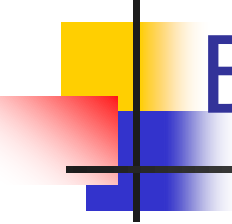
```

// Insert item x into the priority queue
def insert(self, x):
    if self.is_full():
        print("Heap is full")
        return

    self.currentSize += 1
    hole = self.currentSize

    # การหา parent: hole // 2
    while hole > 1 and x < self.array[hole // 2]:
        self.array[hole] = self.array[hole // 2]
        hole //= 2
    self.array[hole] = x

```



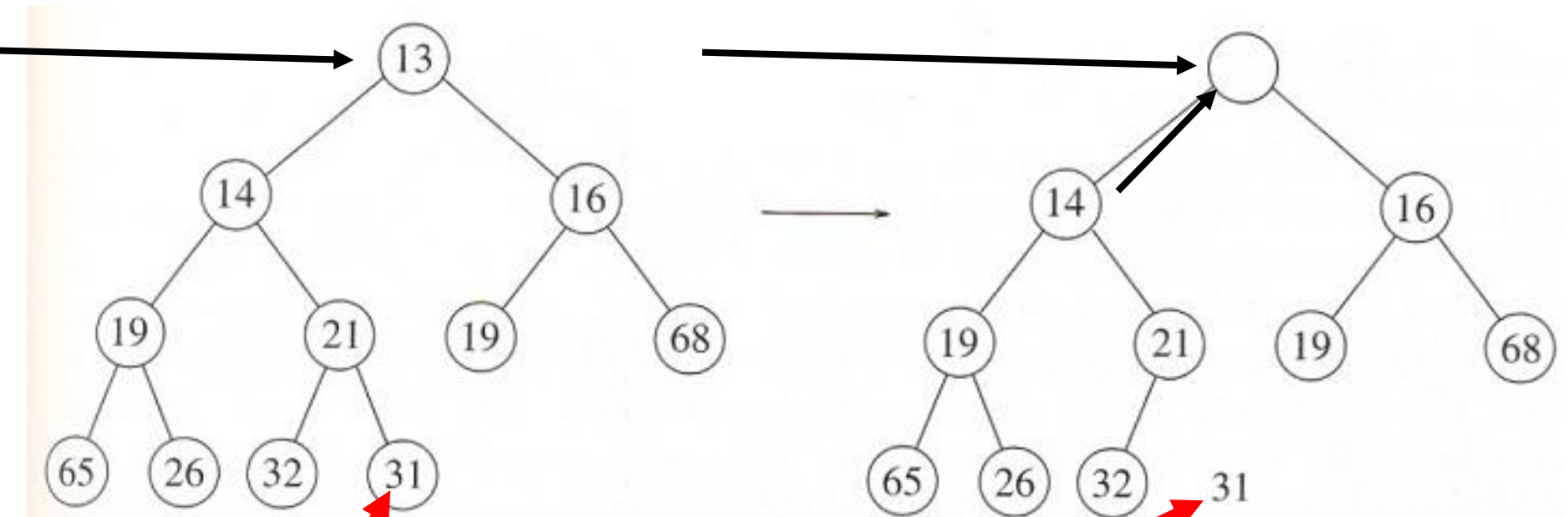
Basic Heap Operations-**deleteMin**

Algorithm

- Find the minimum (at the root)
- Remove the minimum → a hole is created at root
- While the last element X can not be inserted in the hole without violating the heap order
 - Percolate down the hole → swap with smaller child
- Insert X into the hole
- Return minimum

Example

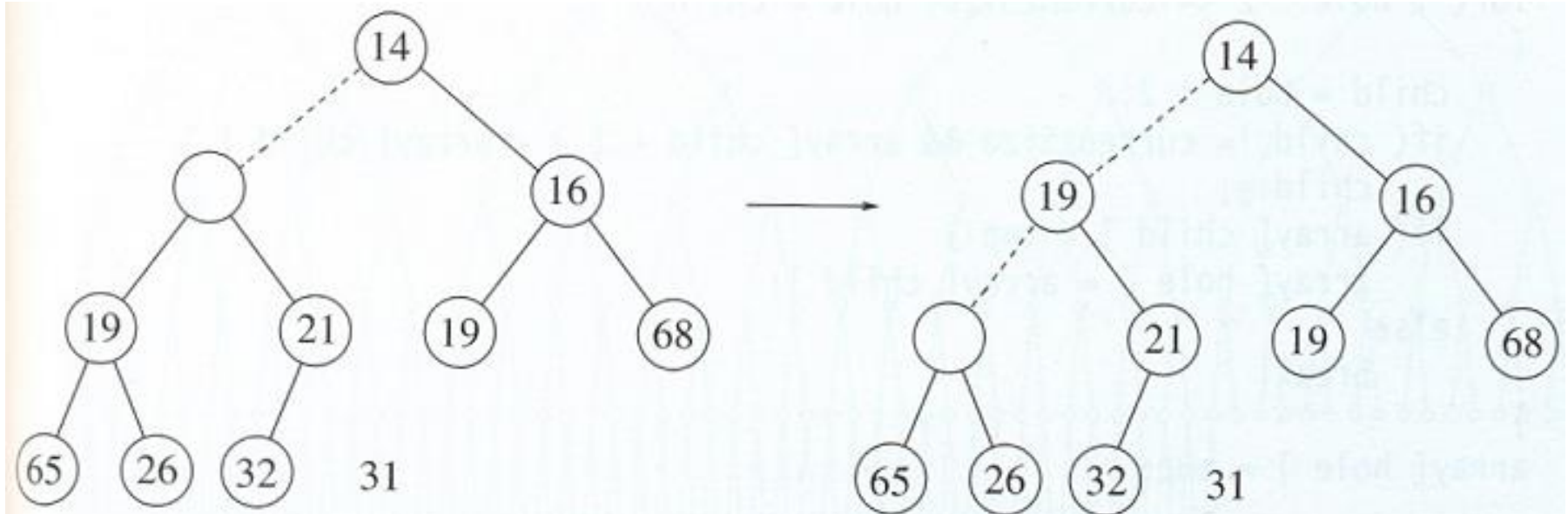
Creation of the hole at the root



First 13 is removed, and 31 is placed in the heap

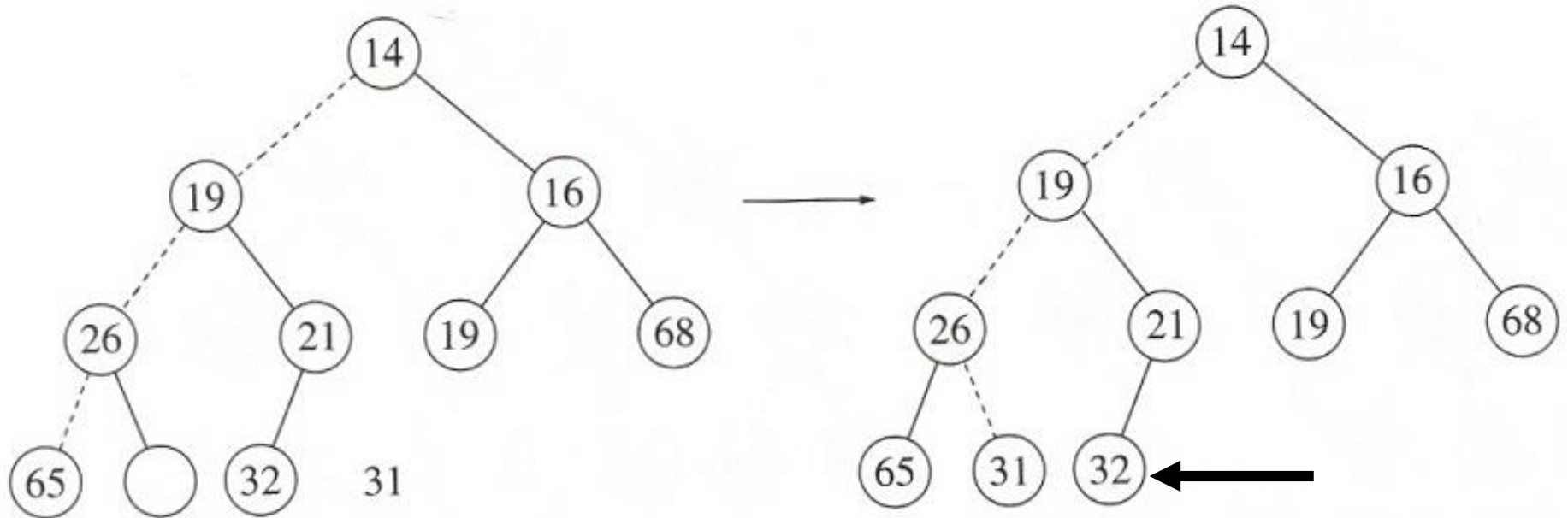
Example

Next two steps in deleteMin



Example

Last two steps in deleteMin



```
// Remove the smallest item from the priority queue
def delete_min(self):
    if self.is_empty():
        print("Heap is empty")
        return
    min_item = self.array[1]
    self.array[1] = self.array[self.currentSize]
    self.currentSize -= 1
    self._percolate_down(1)
    return min_item
def _percolate_down(self, hole):
    temp = self.array[hole]
    while hole * 2 <= self.currentSize:
        child = hole * 2
        if child != self.currentSize and self.array[child + 1] <
self.array[child]:
            child += 1
        if self.array[child] < temp:
            self.array[hole] = self.array[child]
        else:
            break
        hole = child
    self.array[hole] = temp
```